

รายงานผลการออกแบบและจำลองการทำงานวงจร

32x32 Register File (EP_07)

รายวิชา: การออกแบบวงจรดิจิทัลและสถาปัตยกรรมคอมพิวเตอร์ (Beta CPU)

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

ผู้จัดทำ: อัสฎาฐ นิยมพล (รหัสนักศึกษา: 67070502494)

1. วัตถุประสงค์ของการทดลอง

รายงานฉบับนี้จัดทำขึ้นเพื่ออธิบายหลักการทำงานของวงจร 32x32 Register File และแสดงผลการทดลองว่าวงจรที่ออกแบบไว้ทำงานได้ถูกต้อง Register File เป็นส่วนสำคัญของซีพียูแบบ RISC เช่น Unpipelined Beta CPU ที่เรียนในรายวิชานี้ ในการทดลองผู้จัดทำได้ออกแบบวงจรไว้ 2 ระดับ ได้แก่ ระดับโครงสร้างเกต (Gate-level) โดยใช้โปรแกรม JSim และระดับรีจิสเตอร์ทรานสเฟอร์ (RTL) โดยใช้ภาษา VHDL แล้วใช้รูปคลื่นสัญญาณ (Waveform) จากการจำลองการทำงานเป็นหลักฐานยืนยันว่าวงจรทำงานตรงตามที่เรียนมา

2. ทฤษฎีและสถาปัตยกรรมของ 32x32 Register File

Register File ขนาด 32x32 คือหน่วยความจำชั่วคราวภายในซีพียู ประกอบด้วยรีจิสเตอร์ขนาด 32 บิต จำนวน 32 ตัว วงจรนี้มีพอร์ตอ่าน 2 พอร์ต (Read Ports) และพอร์ตเขียน 1 พอร์ต (Write Port) ทำให้ซีพียูอ่านข้อมูล 2 ค่าส่งไปคำนวณที่ ALU และเขียนผลลัพธ์กลับเข้ามาเก็บได้ภายใน 1 รอบสัญญาณนาฬิกา (Clock Cycle) วงจรแบ่งออกเป็น 3 ส่วนหลัก ดังนี้

- Decoder 5-to-32:** รับแอดเดรสเขียน wa (Write Address) ขนาด 5 บิต แล้วแปลงเป็นสัญญาณ Write Enable 32 เส้น ในการออกแบบใช้ NAND4 จำนวน 16 ตัวใน Level 0 และ NOR3 จำนวน 32 ตัวใน Level 1 สัญญาณนี้ทำให้เขียนข้อมูลลงรีจิสเตอร์ได้ครั้งละ 1 ตัวเท่านั้น
- 32-bit D-Register with Enable:** สร้างจาก D-Flip Flop และมี Multiplexer 2-to-1 อยู่ด้านหน้าเพื่อเลือกว่าจะรับค่าใหม่ (wd - Write Data) หรือเก็บค่าเดิมไว้ ขึ้นอยู่กับสัญญาณ Enable นอกจากนี้ตามข้อกำหนดของสถาปัตยกรรม RISC รีจิสเตอร์ตัวที่ 31 (R31) ต้องต่อลงกราวด์ (Hardwired to Zero) ไว้เสมอ
- 32-to-1 Multiplexer:** ใช้เลือกข้อมูลสำหรับพอร์ตอ่าน โดยรับแอดเดรส ra1 และ ra2 ขนาด 5 บิต เพื่อเลือกข้อมูล 32 บิตจากรีจิสเตอร์ 1 ใน 32 ตัวส่งไปที่ rd1 และ rd2 ตามลำดับ โครงสร้างแบ่งเป็นลำดับชั้น (Hierarchical) คือ MUX4 จำนวน 8 ตัวใน Level 0, MUX4 จำนวน 2 ตัวใน Level 1 และ MUX2 จำนวน 1 ตัวในชั้นสุดท้าย

ตารางที่ 1 สรุปองค์ประกอบของ 32x32 Register File

ส่วนประกอบ	หน้าที่	โครงสร้างที่ใช้
Decoder 5-to-32	แปลง wa (5 บิต) เป็นสัญญาณ Write Enable 32 เส้น	NAND4 x16 (Level 0), NOR3 x32 (Level 1)
32-bit D-Register with Enable	เก็บค่าเดิมหรือรับค่าใหม่ wd ตามสัญญาณ Enable	D-Flip Flop + MUX2 หน้าอินพุต, R31 ต่อลงกราวด์
32-to-1 MUX (x2 ชุด)	เลือกข้อมูลจากรีจิสเตอร์ไปยัง rd1 และ rd2	MUX4 x8 (Level 0), MUX4 x2 (Level 1), MUX2 x1

การเขียนข้อมูลเป็นแบบซิงโครนัส คือ wd จะถูกบันทึกลงรีจิสเตอร์ที่ wa ซ้ำๆเมื่อ werf เป็น 1 และเกิดขอบขาขึ้นของ clk ส่วนการอ่านเป็นแบบ Combinational คือเมื่อเปลี่ยนค่า ra1 หรือ ra2 ค่าที่ rd1 หรือ rd2 จะเปลี่ยนตามทันทีโดยไม่ต้องรอสัญญาณนาฬิกา และเนื่องจาก R31 ต่อลงกราวด์ไว้ การอ่านรีจิสเตอร์ตัวนี้จึงได้ค่า 0 เสมอ ซึ่งตรงตามที่สถาปัตยกรรม Beta กำหนด

3. ผลการทดลองในระดับ Gate-level (JSim)

ในส่วนนี้ผู้จัดทำใช้โปรแกรม JSim ตรวจสอบว่าวงจรที่ต่อด้วยลอจิกเกตทำงานได้ตรงตามฟังก์ชันที่ออกแบบไว้หรือไม่

3.1 เงื่อนไขการจำลองการทำงาน (Test Setup)

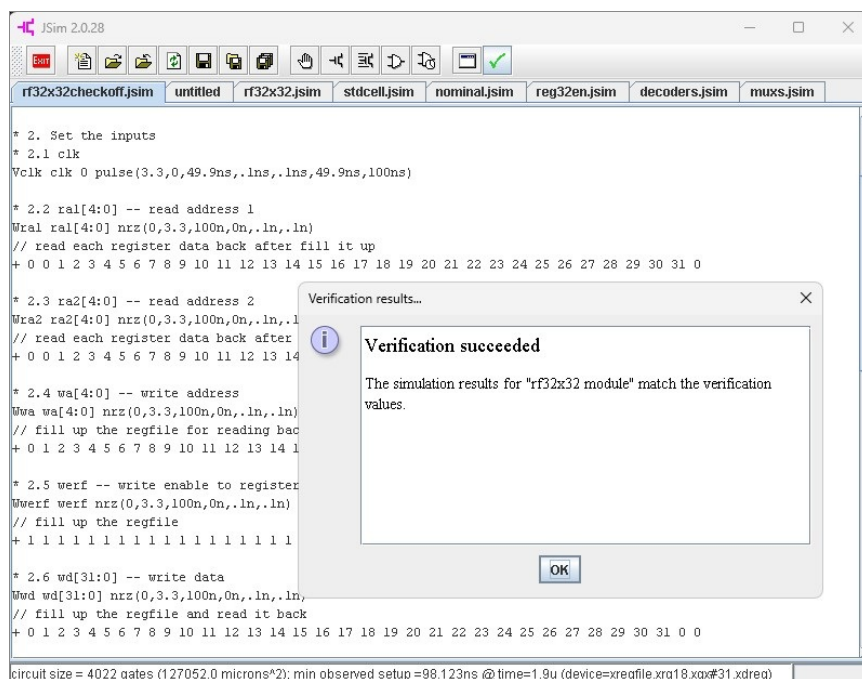
สคริปต์ rf32x32checkoff.jsim กำหนดสัญญาณอินพุตดังตารางที่ 2 โดยช่วงแรกจะเขียนค่าลงรีจิสเตอร์ทีละตัวตั้งแต่ตัวที่ 0 ถึง 31 พร้อมกับให้พอร์ตรอ่านทั้งสองไล่แอดเดรสตามเพื่ออ่านค่ากลับมาตรวจสอบ หลังจากนั้นจึงตั้ง werf เป็น 0 เพื่อปิดการเขียนในช่วงท้าย

ตารางที่ 2 สัญญาณอินพุตของ Testbench ใน JSim

สัญญาณ	ความหมาย	รูปแบบที่ป้อน
clk	สัญญาณนาฬิกา	พัลส์ 0 ถึง 3.3 V คาบ 100 ns (สูง/ต่ำ 49.9 ns)
ra1[4:0], ra2[4:0]	แอดเดรสพอร์ตรอ่าน 1 และ 2	เปลี่ยนทุก 100 ns ไล่ 0 ถึง 31 แล้วกลับเป็น 0
wa[4:0]	แอดเดรสสำหรับเขียน	เปลี่ยนทุก 100 ns ไล่ 0 ถึง 31 แล้วกลับเป็น 0
werf	Write Enable ของ Register File	เป็น 1 ตลอดช่วงเขียน แล้วเป็น 0 ในช่วงท้าย
wd[31:0]	ข้อมูลที่จะเขียน	ไล่ค่า 0 ถึง 31 แล้วเป็น 0 ในช่วงท้าย

3.2 ผลการตรวจสอบฟังก์ชัน (Functional Verification)

เมื่อรันสคริปต์ rf32x32checkoff.jsim พบว่าโปรแกรมแสดงหน้าต่าง "Verification succeeded" ดังรูปที่ 1 แสดงว่าค่าลอจิก (1 และ 0) ที่ได้จากวงจรตรงกับค่าที่ Testbench กำหนดไว้ทุกค่า



รูปที่ 1 ผลการตรวจสอบ rf32x32checkoff.jsim แสดงข้อความ "Verification succeeded"

3.3 การวิเคราะห์เวลา (Timing Analysis)

จากแถบสถานะของ JSim รายงานค่า **min observed setup = 98.123 ns** ในการทดลองนี้ตั้งคาบสัญญาณนาฬิกาไว้ที่ 100 ns ดังนั้นจึงประมาณเวลาหน่วงของวงจร (Critical Path Delay) ได้ดังนี้

$$T_{\text{delay}} = T_{\text{clk}} - T_{\text{setup_observed}}$$

$$T_{\text{delay}} = 100 \text{ ns} - 98.123 \text{ ns} = \mathbf{1.877 \text{ ns}}$$

จะเห็นว่าสัญญาณใช้เวลาผ่านวงจรตั้งแต่ Decoder ไปยัง Register และผ่าน Multiplexer เพียงประมาณ 1.877 ns เท่านั้น จึงอาจลดคาบสัญญาณนาฬิกาลงเหลือประมาณ 2.5 ns ได้โดยวงจรยังทำงานได้เสถียร

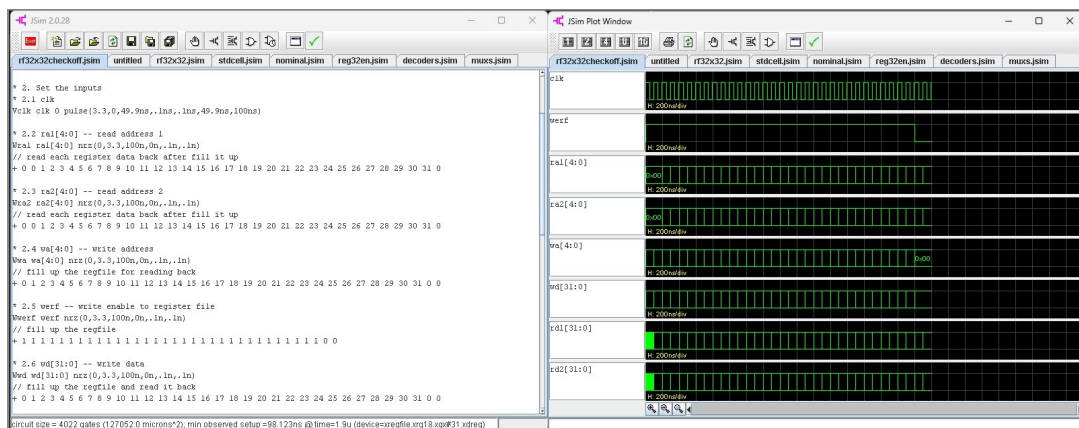
นอกจากนี้แถบสถานะยังแสดงขนาดวงจรเป็น **4022 gates (127052.0 microns²)** ซึ่งส่วนใหญ่มาจากรีจิสเตอร์ 32 บิต จำนวน 32 ตัว และ Multiplexer ของพอร์ตอ่านทั้งสอง สรุปผลของ JSim ได้ดังตารางที่ 3

ตารางที่ 3 สรุปผลการวิเคราะห์ใน JSim

รายการ	ผลที่ได้
ผลการตรวจสอบฟังก์ชัน	Verification succeeded
ขนาดวงจร	4022 เกต (127052.0 microns ²)
min observed setup	98.123 ns
คาบสัญญาณนาฬิกาที่ใช้ทดสอบ	100 ns
Critical Path Delay (คำนวณ)	1.877 ns

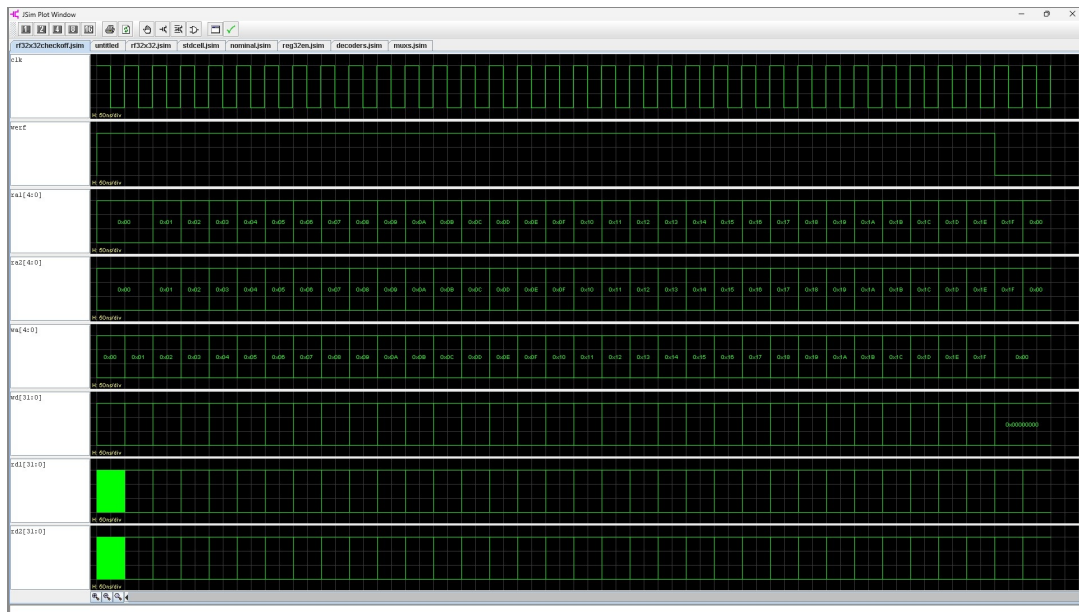
3.4 การวิเคราะห์รูปคลื่นสัญญาณ (Waveform Analysis)

รูปที่ 2 แสดงรูปคลื่นใน JSim Plot Window จะเห็นว่า การเขียนและการอ่านข้อมูลเกิดขึ้นสอดคล้องกับขอบขาขึ้นของสัญญาณนาฬิกา clk



รูปที่ 2 สคริปต์ Testbench (ซ้าย) และ JSim Plot Window (ขวา) แสดงสัญญาณ clk, werf, ra1, ra2, wa, wd, rd1 และ rd2

เมื่อขยายกราฟดังรูปที่ 3 จะเห็นว่าแอดเดรสเปลี่ยนค่าทีละขั้น และค่า rd1, rd2 เปลี่ยนตาม ra1, ra2 ได้ถูกต้องโดยไม่พบสัญญาณรบกวน (Glitch) ที่ส่งผลต่อการทำงานของวงจร



รูปที่ 3 รูปคลื่นสัญญาณ JSim ที่ขยายแล้ว (50 ns/div) แสดงแอดเดรส 0x00 ถึง 0x1F เปลี่ยนทีละขั้น

4. ผลการทดลองในระดับ RTL (VHDL)

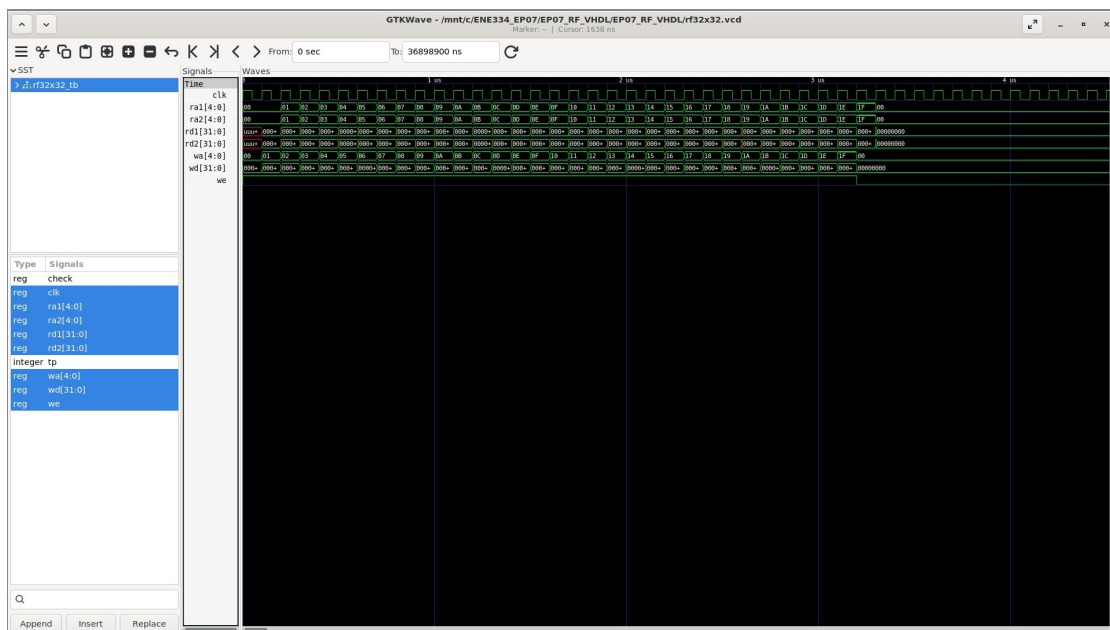
ในส่วนของ VHDL ผู้จัดทำเขียนโค้ดแบ่งเป็น 3 ไฟล์ ได้แก่ dec5to32.vhd, muxs.vhd และ rf32x32.vhd แล้วจำลองการทำงานด้วย GHDL และดูรูปคลื่นด้วย GTKWave

4.1 ขั้นตอนการคอมไพล์และ Testbench

เริ่มจากคอมไพล์ไลบรารีมาตรฐาน (Standard Cell) ก่อน โดยต้องใส่แฟล็ก -fsynopsys เพราะโค้ดเรียกใช้แพ็คเกจ std_logic_textio สำหรับอ่านไฟล์ที่ใช้ทดสอบ จากนั้นทำ Elaboration และรันโมดูล rf32x32_tb ซึ่งผ่านโดยไม่มีข้อผิดพลาดและได้ไฟล์ .vcd ออกมาสำหรับดูรูปคลื่น

4.2 การวิเคราะห์รูปคลื่นสัญญาณ VHDL

รูปที่ 4 เป็นรูปคลื่นจาก GTKWave ที่แสดงเวลาในระดับไมโครวินาที (Microsecond)



รูปที่ 4 รูปคลื่นสัญญาณจาก GTKWave (ไฟล์ rf32x32.vcd) ของ testbench rf32x32_tb

จากรูปสามารถสรุปได้ดังนี้

- **Clock Synchronization:** สัญญาณ clk เป็นพัลส์สม่ำเสมอ ทำหน้าที่กำหนดจังหวะให้ Flip-Flop ทุกตัวในวงจร
- **Address Generation:** ra1, ra2 และ wa เปลี่ยนค่าเป็นเลขฐานสิบหกเรียงจาก 00 ถึง 1F ถูกต้องตามลำดับ
- **Data Consistency:** rd1 และ rd2 ตอบสนองต่อค่าที่เขียนเข้าไป (wd) ตามแอดเดรสของพอร์ตรอ่าน เมื่อซุ่มดู จะเห็นค่าเปลี่ยนทุกรอบนาฬิกาอย่างเป็นระเบียบ และไม่พบสถานะ Undefined (U) หรือ Uninitialized (X) ในช่วงที่ใช้งานจริง ส่วนค่า bvb ที่เห็นช่วงต้นรูปคลื่นของ rd1 และ rd2 เกิดจากรีจิสเตอร์ยังไม่เคยถูกเขียนค่า ตอนเริ่มจำลอง ซึ่งเป็นเรื่องปกติของการจำลอง VHDL

4.3 การเปรียบเทียบผลระหว่าง JSim และ VHDL

รูปคลื่นจากทั้งสองระดับให้พฤติกรรมสอดคล้องกัน ดังสรุปในตารางที่ 4

ตารางที่ 4 เปรียบเทียบผลการจำลองระดับ Gate-level และ RTL

หัวข้อ	Gate-level (JSim)	RTL (VHDL, GHDL + GTKWave)
ลักษณะการออกแบบ	ต่อโครงสร้างเกิดจาก Standard Cell	เขียนโค้ด dec5to32.vhd, muxs.vhd, rf32x32.vhd
แอดเดรส ra1, ra2, wa	เปลี่ยนจาก 0x00 ถึง 0x1F	เปลี่ยนจาก 00 ถึง 1F
สัญญาณเขียน	werf เป็น 1 แล้วลงเป็น 0 ในช่วงท้าย	we เป็น 1 แล้วลงเป็น 0 ในช่วงท้าย
ค่า wd ในช่วงท้าย	0x00000000	00000000
หลักฐานความถูกต้อง	Verification succeeded	รูปคลื่นไม่พบสถานะ U/X ในช่วงใช้งาน

5. สรุปผลการทดลอง

จากการทดลองพบว่าวงจร 32x32 Register File ที่ออกแบบทั้งระดับเกตใน JSim และระดับ VHDL ทำงานได้ตรงตามสเปกของสถาปัตยกรรม Beta CPU ผลการจำลองทั้งสองแบบให้ผลการอ่านและเขียนข้อมูลตรงกัน ทั้งด้านฟังก์ชันและด้านเวลา แสดงว่าการออกแบบและการเชื่อมต่อสายสัญญาณ (Structural Mapping) ถูกต้อง และสามารถนำ Register File นี้ไปใช้ประกอบเป็นซีพียูในบทเรียนถัดไปได้